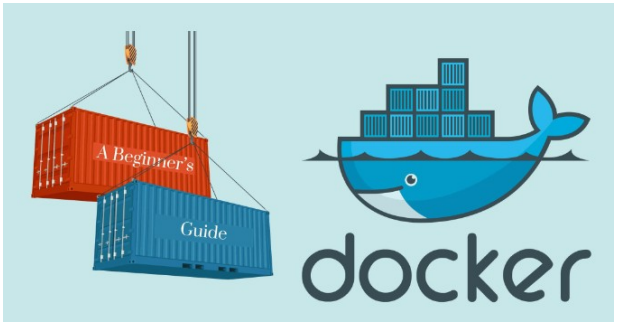


Administración de Sistemas Operativos

UD – 2

Docker

Contenidos





Índice

1. IMÁGENES Y CONTENEDORES....3
¿Qué es una máquina Virtual ?....3
¿Dónde se almacenan imágenes, contenedores y datos?....3
2.- REGISTRO: DOCKER HUB....4
....4
4.- Ejercicios....4
4.- CREANDO Y ARRANCANDO CONTENEDORES CON "DOCKER RUN"....5
¿Qué hacen los principales comandos?....5
¿Y los principales comandos?....5
5.- Ejecutando contenedores....7
5.2 .- Ejecución de servicios: puertos y variables de entorno....8
5.3. Ejecutar órdenes en los contenedores....8
5.4. Obtención de información de los contenedores....9
5.4.1 docker ps....9
5.4.2 docker inspect....10
5.4.3 docker logs....10
5.5. Gestión de contenedores....10

1. IMÁGENES Y CONTENEDORES

¿Qué es una máquina Virtual ?

Antes de comenzar, es importante aclarar algunos conceptos sobre qué son imágenes y contenedores y cuáles son sus características. Algunos conceptos a tener en cuenta son:

- **Imágenes**:
  - La imagen es una plantilla de solo lectura que se utiliza para crear contenedores. A partir de una imagen pueden crearse múltiples contenedores.
  - Las imágenes, además de tener su sistema de ficheros predefinido, tienen una serie de parámetros predefinidos (comandos, de variables de entorno, etc.) con valores por defecto y que se pueden personalizar en el momento de crear el contenedor.
  - Docker permite crear nuevas imágenes basándose en imágenes anteriores. Se podría decir que una imagen puede estar formada por un conjunto de "capas" que han modificado una imagen base.
    - Al crear una nueva imagen, simplemente estamos añadiendo una capa a la imagen anterior, la que actúa como base.
- **Contendores**:
  - Son instancias de una imagen.
  - Pueden ser arrancados, parados y ejecutados.
  - Cada contenedor Docker posee un **identificador único de 64 caracteres,** pero habitualmente se utiliza una **versión corta con los primeros 12 caracteres**.
    - Los comandos Docker habitualmente soportan ambas versiones.
  - También podemos referimos a los contenedores con menos caracteres, siempre que su expresión regular sea única.
    - Por ejemplo, si tienes dos contenedores y el ID de un contenedor empieza por "7" y el ID del otro por "9", simplemente escribiendo como ID "7" se referirá al primer contenedor, ya que no hay otro contenedor que empiece por "7". Si hubiera varios contenedores que empezaran igual, habría que añadir más caracteres hasta encontrar un patrón no común. Un símil para entender estos conceptos: una instalación de una distribución de Linux mediante un DVD. Ese DVD sería nuestra imagen y el sistema operativo instalado sería el contenedor. Detallaremos conceptos relacionados sobre la creación de imágenes en futuras unidades

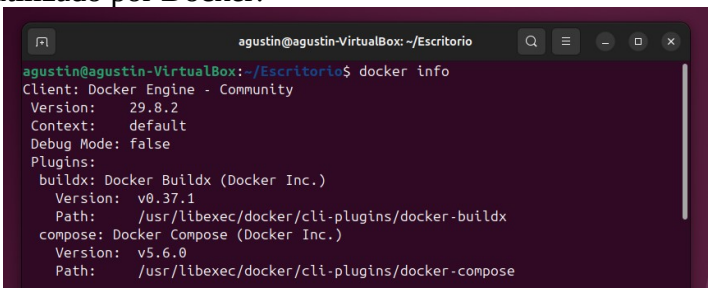
Un símil para entender estos conceptos: una instalación de una distribución de Linux mediante un DVD. Ese DVD sería nuestra imagen y el sistema operativo instalado sería el contenedor.

¿Dónde se almacenan imágenes, contenedores y datos?

El lugar donde se almacenan contenedores e imágenes puede variar según distribución/sistema operativo, driver de almacenamiento y versión de Docker. Normalmente, mediante el siguiente comando de Docker, podemos ver información del sistema, incluyendo el directorio de Docker.

docker info

Ese comando nos ofrece información sobre el estado de Docker. Para conocer donde se almacena la información, dos datos son importantes: directorio de Docker y driver de almacenamiento. Driver de almacenamiento utilizado por Docker:



En el ejemplo se nos indica el driver de almacenamiento utilizado (“overlay2”) y sobre qué sistema de archivos está funcionando (“extfs”, en concreto “ext4”). Para saber más sobre los distintos drivers de almacenamiento de Docker, podéis consultar :

https://docs.docker.com/storage/storagedriver/select-storage-driver/

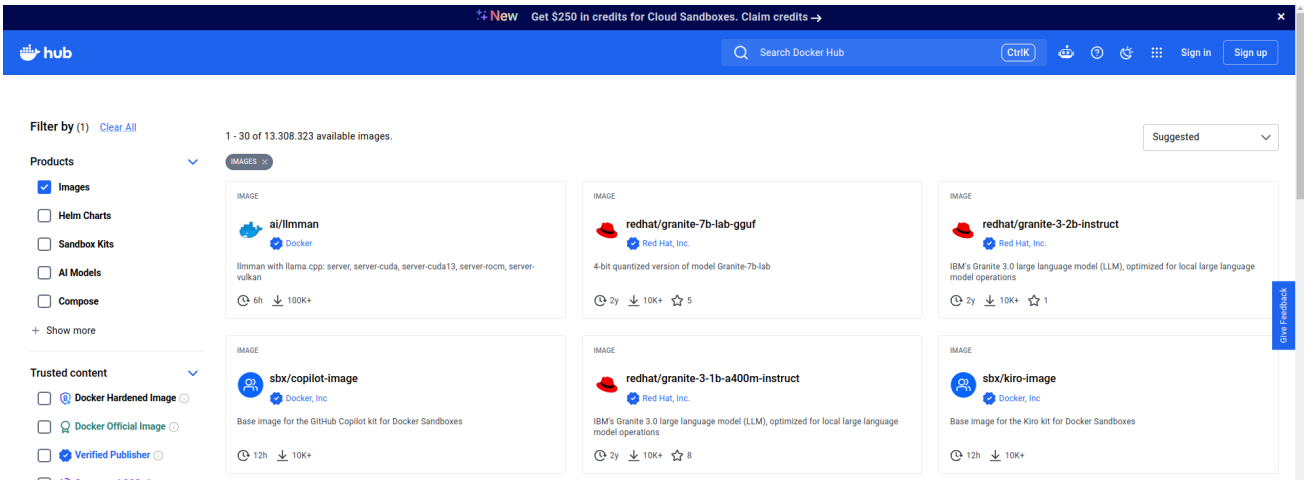
Directorio de Docker: Directorio donde se almacena todo lo relacionado con Docker. Al utilizar el driver “overlay2” sabemos que

- La información de las **imágenes** se encuentra en “**/var/lib/docker/overlay2”**.
  - Recordemos que las imágenes se basan en capas, una imagen puede estar creada por un conjunto de capas.
  - El almacenamiento temporal de contenedores (es decir, pequeños cambios en los contenedores), se realiza como “versiones de las imágenes”, es decir, una capa más de la imagen base.
- La configuración de los **contenedores** se almacena en “**/var/lib/docker/containers**”.
  - Para acceso a datos compartidos, persistencia, etc. existe la figura de los volúmenes, la cual detallaremos en posteriores unidades.

2.- REGISTRO: DOCKER HUB

Docker Hub es una “plataforma de registro” de Docker. Los servicios básicos son gratuitos y nos permite registrar imágenes Docker, haciéndolas públicas o privadas. Contiene un gran ecosistema de imágenes ya creadas, usualmente con instrucciones de instalación y uso, además de un buscador que nos permite encontrar imágenes según distintos parámetros. Enlace al buscador

https://hub.docker.com/search?q=&type=image



4.- Ejercicios

- Crea una cuenta personal de DockerHub
- Busca el docker último de ubuntu

4.- CREANDO Y ARRANCANDO CONTENEDORES CON “DOCKER RUN”

¿Qué hacen los principales comandos?

Es posiblemente el comando “docker run” más utilizado. Podríamos decir que este comando **crea un contenedor a partir de una imagen y lo arranca**. Importante: un error común es creer que “docker run” solo arranca contenedores. Si haces varios “docker run”, estás creando varios contenedores, no arrancando varias veces un contenedor.

docker run ....

Para crear un contenedor sin arrancarlo (recordamos, “docker run” crea y arranca), existe el comando

docker create ....

Mediante el comando “docker ps” podemos listar los contenedores en ejecución en el sistema. Si ejecutamos el siguiente comando:

docker ps

O si tenemos contenedores en ejecución, podemos obtener algo similar utilizando la opción -a. Obtendremos un listado de todos los contenedores, tanto aquellos en funcionamiento como aquellos que están parados.

docker ps -a

La información que obtenemos de los contenedores es la siguiente:

- **CONTAINER_ID:** identificador único del contenedor (versión 12 primeros caracteres).
- **IMAGE**: imagen utilizada para crear el contenedor.
- **COMMAND**: comando que se lanza al arrancar el contenedor.
- **CREATED**: cuando se creó el contenedor.
- **STATUS**: si el contenedor está en marcha o no (indicando cuánto lleva en marcha o cuánto hace que se paró).
- **PORTS**: redirección de puertos del contenedor (lo veremos más adelante en la unidad).
- **NAMES**: nombre del contenedor. Se puede generar como parámetro al crear el contenedor, o si no se indica nada, el propio Docker genera un nombre aleatorio.

¿Y los principales comandos?

Si te has fijado, el uso de docker consiste en pasar a este una cadena de opciones y comandos seguida de argumentos. La sintaxis adopta esta forma:
docker [option] [command] [arguments]

Si desea ver las opciones disponibles para un comando específico, escriba lo siguiente:

docker <*comando*> --help

Para ver información sobre Docker relacionada con todo el sistema, utilice lo siguiente:

docker info

Y los principales comandos son:

- **attach** -> Attach local standard input, output, and error streams to a running container
- **build** -> Build an image from a Dockerfile
- **commit** -> Create a new image from a container's changes
- **compose** (plugin) -> Define and run multi-container applications with Docker
- **container** -> Manage containers
- **context** -> Manage contexts
- **cp** -> Copy files/folders between a container and the local filesystem
- **create** -> Create a new container
- **diff** -> Inspect changes to files or directories on a container's filesystem
- **events** -> Get real time events from the server
- **exec** -> Run a command in a running container
- **export** -> Export a container's filesystem as a tar archive
- **history** -> Show the history of an image
- **image** -> Manage images
- **images** -> List images
- **import** -> Import the contents from a tarball to create a filesystem image
- **info** -> Display system-wide information
- **inspect** -> Return low-level information on Docker objects
- **kill** -> Kill one or more running containers
- **load** -> Load an image from a tar archive or STDIN
- **login** -> Log in to a Docker registry
- **logout** -> Log out from a Docker registry
- **logs** -> Fetch the logs of a container
- **manifest** -> Manage Docker image manifests and manifest lists
- **network** -> Manage networks
- **pause** -> Pause all processes within one or more containers
- **plugin** -> Manage plugins
- **port** -> List port mappings or a specific mapping for the container
- **ps** -> List containers
- **pull** -> Pull an image or a repository from a registry
- **push** -> Push an image or a repository to a registry
- **rename** -> Rename a container
- **restart** -> Restart one or more containers
- **rm** -> Remove one or more containers
- **rmi** -> Remove one or more images
- **run** -> Run a command in a new container
- **save** -> Save one or more images to a tar archive (streamed to STDOUT by default)
- **search** -> Search the Docker Hub for images
- **secret** -> Manage Swarm secrets
- **service** -> Manage Swarm services
- **stack** -> Manage Swarm stacks
- **start** -> Start one or more stopped containers
- **stats** -> Display a live stream of container(s) resource usage statistics
- **stop** -> Stop one or more running containers
- **swarm** -> Manage Swarm
- **system** -> Manage Docker
- **tag** -> Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
- **top** -> Display the running processes of a container
- **trust** -> Manage trust on Docker images
- **unpause** -> Unpause all processes within one or more containers
- **update** -> Update configuration of one or more containers
- **version** -> Show the Docker version information
- **volume** -> Manage volumes
- **wait** -> Block until one or more containers stop, then print their exit codes

5.- Ejecutando contenedores

Tras instalar Docker y comprobar que todo funciona correctamente, el siguiente paso es empezar a ejecutar contenedores. Necesitamos las imágenes de aquello que queremos probar. Se descargan así:

docker pull nombre_imagen:version

***Descarga la versión indicada o latest si no se especifica.*

Tarea

Descargar:

- Ubuntu (última versión)
- Httpd (última versión)
- Mariadb (última versión)
- MySQL 5.7

Solución

- *docker pull ubuntu:latest*
- *docker pull httpd:latest*
- *docker pull mariadb:latest*
- *docker pull mysql:5.7*

La orden fundamental para ejecutar contenedores es docker run, cuya función es poner en ejecución un contenedor a partir de una imagen de referencia.

Importante: si la imagen no existe localmente, docker run la descargará automáticamente desde Docker Hub.

docker run tiene una sintaxis sencilla pero multitud de opciones (alrededor de 100). Estas son las más útiles para el día a día:

- **-d / --detach** -> Ejecuta el contenedor en background (servicios)
- **-e / --env** -> Establece variables de entorno
- **-h / --hostname** -> Nombre de red del contenedor
- **--help** -> Ayuda de las opciones
- **-i / --interactive** -> Mantiene STDIN abierta
- **--ip** -> Asigna una IP concreta
- **--name** -> Nombre del contenedor
- **--net / --network** -> Conecta el contenedor a una red
- **-p / --publish** -> Mapea puertos host:contenedor
- **--restart** -> Política de reinicio si el contenedor se cae
- **--rm** -> Destruye el contenedor al pararlo
- **-t / --tty** -> Permite acceso a un terminal
- **-u / --user** -> Usuario de ejecución
- **-v / --volume** -> Monta un bind mount o volumen
- **-w / --workdir** -> Directorio de trabajo

Ejemplo 1: *Ubuntu interactivo*

docker pull ubuntu:20.04
docker run -it ubuntu:20.04 /bin/bash

Resultado:

root@ef2bea1d6cb1:/#

Ejemplo 2: *MySQL con contraseña root*

docker pull mysql

docker run --name EquipoMysql -e MYSQL_ROOT_PASSWORD=usuario -d mysql

- *EquipoMysql → nombre del contenedor*
- *usuario → contraseña del root de MySQL*
- *mysql → etiqueta de la versión*

Para obtener un shell bash dentro del contenedor:

docker exec -it EquipoMysql bash

Los logs se consultan con:

docker logs EquipoMysql

Notas importantes

- Usa -it al hacer docker run si el contenedor no tiene servicio (shell interactivo). -it = -i + -t: abre STDIN y permite un terminal.
- No añadas al final una orden distinta de /bin/bash (u otro shell) en contenedores con servicio: sobrescribirías su orden de arranque y podrías dejarlos inservibles.
- Los contenedores sin servicio se paran al salir de ellos. Para reiniciarlos: docker start.
- Para contenedores con servicio hay que tener en cuenta varios aspectos que vemos a continuación.

5.2 .- Ejecución de servicios: puertos y variables de entorno

- -d: ejecuta el servicio en background (detach). Si no lo usas, el terminal se bloquea mostrando los logs y tendrás que salir con Ctrl+C (lo que para el contenedor).
- -p host:contenedor: redirige puertos para que el servicio sea accesible desde el exterior. Normalmente se usa el puerto por defecto del servicio. Puedes añadir varias reglas -p, pero no puedes tener dos servicios escuchando en el mismo puerto del host.
- -e VAR=valor: define variables de entorno. Las importantes vienen descritas en la página de cada imagen en Docker Hub.

Ejemplo :
Servidor Apache sin -d ni redirección (bloquea el terminal mostrando logs):

docker run httpd

Servidor Apache en background, accesible desde el puerto 8888:

docker run -d -p 8888:80 httpd

MySQL accesible por el puerto 3306 con contraseña root:

docker run -d -p 3306:3306 --name EquipoMysql -e MYSQL_ROOT_PASSWORD=usuario mysql

5.3. Ejecutar órdenes en los contenedores

Con los contenedores en ejecución querremos ejecutar órdenes dentro de ellos: instalar paquetes, modificar ficheros, habilitar módulos, etc. Esto se hace con docker exec (el contenedor debe estar en ejecución).

Sintaxis:

docker exec [opciones] nombre_contenedor orden [argumentos]

Opciones más importantes:

- -it: interactividad con shell (/bin/bash normalmente).
- -u / --user: ejecutar como otro usuario distinto de root.
- -w / --workdir: ejecutar desde un directorio concreto.

Ejemplo 1: *Obtener un terminal en un contenedor Apache llamado web:*

docker run -dit --name web -p 8080:80 httpd
docker exec -it web /bin/bash
root@5d96ce1f7374:/usr/local/apache2#

Ejemplo 2: *Mostrar contenido de una carpeta (sin interactividad, no hace falta -it):*

docker exec web ls /usr/local/apache2/htdocs

Ejemplo 3: *Crear un fichero "HOLA MUNDO" (órdenes compuestas con sh -c):*

docker exec -it web sh -c "echo 'HOLA MUNDO' >> /usr/local/apache2/htdocs/index.html"

Ejemplo 4: *docker cp — copiar ficheros entre host y contenedor*
Del host al contenedor:

docker cp prueba.html web:/usr/local/apache2/htdocs/index.html

Del contenedor al host:

docker cp web:/usr/local/apache2/htdocs/index.html ./test.html

Nota: los contenedores vienen con lo mínimo instalado. Si necesitas instalar algo, normalmente antes hay que hacer apt update (la mayoría están basados en Debian).

Ejemplo 5:
Arrancar un contenedor ubuntu:18.04 llamado ubuntu y:

1. Salir del contenedor.
2. Rearrancarlo desde tu equipo.
3. Sin entrar, mostrar el fichero /etc/os-release y capturar un pantallazo.

docker run -it --name modulo23 ubuntu:18.04 /bin/bash
docker ps -a
docker start modulo23
docker exec modulo23 cat /etc/os-release

Ejemplo 5: *Apache con PHP*

docker run -d --name web -p 8181:80 php:7.4-apache
docker exec web sh -c "echo '<h1>Hola soy Agsutin Aguilera</h1>' >> /var/www/html/index.html"
docker cp info.php web:/var/www/html/

5.4. Obtención de información de los contenedores

Con el uso de Docker el número de contenedores crece y llega un momento en que no recordamos detalles: si tenía persistencia, redirecciones, nombre, etc. Para eso tenemos:

- docker ps
- docker inspect
- docker logs

5.4.1 docker ps

Muestra información de los contenedores. Menos exhaustiva que inspect, pero útil para ver:

- Estado (EXITED parado / UP funcionando).
- Imagen de la que deriva.
- Tamaño actual.
- ENTRYPOINT (orden que ejecuta al arrancar).
- Nombre del contenedor (dado por nosotros o por Docker).
- Fecha de creación.
- Redirecciones de puertos (solo si está en ejecución).

Ejemplo :

```
docker ps                                  # contenedores en ejecución
docker ps -a                               # todos (parados y en ejecución)
docker ps -a -s                            # con tamaño
docker ps -l                               # el último creado
docker ps --filter name=servidor_web
docker ps --filter publish=8080            # contenedores que exponen el 8080
También existe --format para personalizar la salida.
```

5.4.2 docker inspect

Información detallada en formato JSON. Se puede consultar por nombre o por ID:

```
docker inspect jenkins
docker inspect 5e5adf6815bc
```

Devuelve datos como: ID, puertos abiertos y redirecciones, bind mounts y volúmenes, tamaño, configuración de red, ENTRYPOINT, variables de entorno, etc.

Formatear la salida con Go Templates (--format / -f):

```
docker inspect --format 'La IP es {{.NetworkSettings.Networks.bridge.IPAddress}}' jenkins
# La IP es 172.17.0.2

docker inspect --format 'Las redirecciones son {{.NetworkSettings.Ports}}' jenkins
# Las redirecciones son map[50000/tcp:[{0.0.0.0 50000}] 8080/tcp:[{0.0.0.0 9090}]]
```

Para usar --format hay que conocer la estructura del JSON que devuelve inspect.

5.4.3 docker logs

Muestra lo que está pasando dentro del contenedor (tanto parados como en ejecución):

```
docker logs jenkins
docker logs 5e5adf6815bc

docker logs -f jenkins          # sigue la salida en tiempo real
docker logs --tail 5 jenkins    # últimas 5 líneas
```

Consultar los logs es especialmente útil cuando un contenedor con servicio está fallando.

5.5. Gestión de contenedores

Con el tiempo surgen necesidades como:

- Parar un contenedor que ya no usamos o que ocupa un puerto.
- Eliminar contenedores que ya no necesitamos.
- Iniciar un contenedor parado.
- Reiniciar un contenedor para aplicar nueva configuración.

Órdenes principales:

- **docker stop** -> Detiene un contenedor (por nombre o ID)
- **docker rm** -> Borra un contenedor
- **docker start** -> Inicia un contenedor parado
- **docker restart** -> Reinicia un contenedor en ejecución

Ejemplo

Parar

```
docker stop servidorWeb
docker stop -t 10 ea9b922190d8      # espera 10s antes de matar
```

Eliminar

```
docker rm servidorBD
docker rm -f jenkins                # fuerza aunque esté en ejecución
docker rm $(docker ps -a -q)        # elimina TODOS los contenedores
```

Iniciar

```
docker start jenkins
docker start -i jenkins             # inicia e interactúa (attach STDIN)
```

Reiniciar

```
docker restart ea9b922190d8
docker restart -t 10 ea9b922190d8   # espera 10s
```

6.-Gestión de Almacenamiento: Volúmenes y Bind Mounts

La clave para entender el almacenamiento en Docker gira en torno a una pregunta fundamental: ¿qué pasa con los datos si borro el contenedor? Por defecto, cualquier dato escrito dentro de un contenedor se pierde cuando este se destruye. Para solucionarlo, Docker ofrece dos mecanismos principales de persistencia:

- volúmenes (volumes)
- montajes de enlace (bind mounts).
- *En este apartado usaremos la imagen httpd (Apache) como hilo conductor.*

¿Por qué httpd necesita almacenamiento persistente?



El directorio raíz del sitio web en la imagen oficial de httpd se encuentra en /usr/local/apache2/htdocs/ dentro del contenedor. Si ejecutas docker run -d -p 8080:80 httpd, solo verás la página de bienvenida por defecto. En cuanto elimines ese contenedor, cualquier página que hayas metido manualmente desaparecerá.

En un escenario real de administración de sistemas, el contenido del sitio web debe existir independientemente del ciclo de vida del contenedor. Para ello, se "monta" un directorio del host (o un volumen gestionado por Docker) dentro de la ruta raíz del servidor Apache, de modo que este lea los ficheros desde fuera del contenedor.

6.1. Bind Mounts (Montajes de enlace)

Un bind mount consiste en mapear un directorio concreto que tú eliges en el host a una ruta dentro del contenedor. Es la forma más intuitiva durante el desarrollo: editas el fichero en el host con tu editor y, al refrescar el navegador, ves el cambio al instante.

Ejemplo con httpd:

Prepara el directorio y el fichero web en el host:

```
mkdir -p ~/mi-web
echo "<h1>Hola desde ASIR</h1>" > ~/mi-web/index.html
```

Ejecuta el contenedor httpd montando el directorio del host en la raíz web del contenedor:

```
docker run -d --name web-bind -p 8080:80 \
  -v ~/mi-web:/usr/local/apache2/htdocs/ \
  httpd:latest
```

Verifica: accede a http://localhost:8080 y deberías ver "Hola desde ASIR".

Punto clave: el bind mount "presta" un directorio del host al contenedor, lo que genera un alto acoplamiento. Es adecuado para desarrollo, pero en producción, si el directorio del host se borra por error o los permisos no son correctos, el contenedor falla de inmediato.

Además, la imagen oficial de httpd se ejecuta con un usuario concreto, así que al montar directorios del host hay que vigilar que los permisos permitan al proceso del contenedor leerlos.



6.2 Volumes - Volúmenes

Un volumen es un área de almacenamiento gestionada por el propio Docker. Tú solo le das un nombre y Docker le asigna espacio en el host (en Linux, bajo /var/lib/docker/volumes/). El contenedor lo usa por su nombre, sin que tengas que preocuparte por la ruta concreta en el host.

Ejemplo con httpd:

Crea un volumen con nombre:

```
docker volume create web_content
```

Ejecuta el contenedor httpd montando el volumen en la raíz web:

```
docker run -d --name web-vol -p 8081:80 \
  -v web_content:/usr/local/apache2/htdocs/ \
  httpd:latest
```

Introduce el fichero web en el volumen (puedes hacerlo con un contenedor temporal):

```
docker run --rm -v web_content:/data alpine \
  sh -c "echo '<h1>Contenido persistente</h1>' > /data/index.html"
```

Verifica: accede a http://localhost:8081.

Punto clave: los volúmenes son el mecanismo de persistencia recomendado por Docker. Tanto los datos de contenido de servicios sin estado como httpd, como los ficheros de bases de datos de servicios con estado como MySQL, deberían usar volúmenes con nombre. Al eliminar un contenedor con el parámetro -v se borran también los volúmenes anónimos, pero los volúmenes con nombre no se eliminan automáticamente, lo que protege mejor los datos.



¿Qué esta pasando en esta línea?

```
docker run --rm -v web_content:/data alpine \
  sh -c "echo '<h1>Contenido persistente</h1>' > /data/index.html"
```

El problema que estamos resolviendo es el siguiente: Tienes un volumen (web_content) vacío y necesitas meter un archivo dentro, pero no tienes ninguna forma sencilla de acceder a él desde el sistema anfitrión (host).

En lugar de arrancar el contenedor de Apache (httpd) y entrar con docker exec para crear el archivo a mano, lo que hacemos es usar un **contenedor desechable (Alpine)** para hacer esa tarea de forma rápida y automatizada.

1. **docker run**: **Arranca** un nuevo contenedor.
2. **--rm:** Esta es la clave. Le dice a Docker que **destruya el contenedor** en el momento en que termine de ejecutar la orden. Es un contenedor "de usar y tirar". No queremos que se quede ahí ocupando espacio.
3. **-v web_content:/data: Conecta el volumen** llamado web_content (que creaste antes) a la carpeta /data dentro de este contenedor temporal.
4. **alpine**: Es la **imagen que usamos**. ¿Por qué Alpine? Porque es una distribución de Linux ultra ligera (pesa apenas unos megas). Si usáramos una imagen como ubuntu (que pesa mucho más) solo para escribir un archivo de texto, sería una pérdida de tiempo y recursos. Alpine es perfecta para tareas rápidas en segundo plano.
5. **sh -c "...":** Aquí le decimos a Alpine qué debe hacer una vez que arranque. Le pedimos que **abra una consola** (**sh**) y **ejecute** el comando que está entre comillas (**-c**).
6. **echo '<h1>Contenido persistente</h1>' > /data/index.html:** Este es el **comando real**. Escribe el texto HTML y lo guarda (con el símbolo >) en el archivo index.html dentro de la carpeta /data. Como esa carpeta /data está conectada al volumen web_content, el archivo se guarda realmente en el volumen.

6.3 Cuadro comparativo

Dimensión	Bind Mount	Volume
Ruta en el host	La eliges tú	La gestiona Docker
Visibilidad de los datos	Visible y editable directamente desde el host	Se accede mediante comandos Docker o desde el contenedor
Datos tras borrar el contenedor	Se conservan (en el directorio del host)	Se conservan (el volumen existe de forma independiente)
Portabilidad	Baja (depende de rutas del host)	Alta (basta con el nombre del volumen para migrar)
Uso típico en ASO	Desarrollo y depuración, montaje de ficheros de configuración	Persistencia en producción, almacenamiento de bases de datos

7.- DockerFile – Creación de Imágenes propias

Hasta ahora hemos trabajado con imágenes descargadas de Docker Hub (ubuntu, httpd, mariadb, mysql…) y hemos creado contenedores a partir de ellas. En los apartados anteriores vimos cómo ejecutar un Apache (httpd) y cómo modificar su contenido con docker exec o docker cp. Sin embargo, esas modificaciones se pierden si el contenedor se destruye. El **Dockerfile** nos permite crear una imagen propia de Apache que ya incluya nuestra web, nuestra configuración y todo lo necesario, de forma reproducible.

7.1. ¿Qué es un Dockerfile?

Un Dockerfile es un archivo de texto plano que contiene una serie de instrucciones que Docker ejecuta de forma secuencial para construir una imagen de contenedor. Cada instrucción crea una nueva capa en la imagen final, aprovechando el sistema de capas que ya conocemos de las imágenes base.

En el caso de httpd, el Dockerfile nos permitirá, por ejemplo, partir de la imagen oficial httpd:latest y añadir nuestra propia página index.html, un info.php, una configuración personalizada de Apache o los módulos que necesitemos.

7.2. Instrucciones fundamentales

Aunque existen muchas instrucciones disponibles, estas son las más utilizadas en el día a día:

- **FROM**: Define la imagen base sobre la que se construirá la nueva imagen. Es obligatoria y debe ser la primera instrucción (excepto comentarios o directivas especiales). En nuestro caso, lo habitual será FROM httpd:latest o FROM httpd:2.4-alpine para reducir el tamaño final.

  Ejemplo: FROM httpd:latest

- **WORKDIR**: Establece el directorio de trabajo para las instrucciones posteriores (RUN, CMD, ENTRYPOINT, COPY, etc.) y para el contenedor que se ejecute a partir de la imagen.

  Ejemplo: WORKDIR /usr/local/apache2/htdocs.

- **COPY**: Copia archivos o directorios desde el contexto de construcción (el directorio donde está el Dockerfile y sus subdirectorios) al sistema de archivos de la imagen. Es la instrucción clave para incorporar nuestra web a la imagen de Apache.

  Ejemplo: COPY index.html /usr/local/apache2/htdocs/ o COPY . /usr/local/apache2/htdocs/.

- **RUN**: Ejecuta comandos durante la **construcción** de la imagen. Se usa típicamente para instalar paquetes, crear directorios o modificar permisos. Cada RUN genera una nueva capa.

  Ejemplo: RUN sed -i 's/Listen 80/Listen 8080/' /usr/local/apache2/conf/httpd.conf.

- **EXPOSE**: Documenta el puerto en el que el contenedor escuchará en tiempo de ejecución. En httpd el puerto por defecto es el 80. No publica el puerto por sí misma; para ello se usa -p al ejecutar docker run.

  Ejemplo: EXPOSE 80.

- **CMD**: Define el comando por defecto que se ejecutará al iniciar un contenedor a partir de la imagen. Solo puede haber una instrucción CMD efectiva (la última). En la imagen oficial de httpd, el CMD ya está definido para arrancar Apache en primer plano (httpd-foreground), por lo que normalmente **no es necesario** sobrescribirlo. Puede sobrescribirse fácilmente pasando un comando a docker run.

- **ENTRYPOINT**: Configura el contenedor para que se ejecute como un ejecutable. A diferencia de CMD, los argumentos pasados a docker run se añaden al comando definido en ENTRYPOINT en lugar de reemplazarlo. En la imagen oficial de httpd ya viene definido (/usr/local/bin/httpd-foreground), así que generalmente no lo tocamos.

7.3. Buenas prácticas en la escritura de Dockerfiles

Escribir un Dockerfile funcional es solo el primer paso; optimizarlo marca la diferencia:

- **Aprovechar la caché de capas**: Docker reutiliza capas de construcciones anteriores si las instrucciones no han cambiado. Para maximizar los aciertos, ordena las instrucciones de menor a mayor frecuencia de cambio. Por ejemplo, copia primero la configuración de Apache (que cambia poco) y **después** el contenido web (index.html, imágenes, etc.), que cambia con más frecuencia.
- **Minimizar el número de capas**: Combina comandos relacionados en una sola instrucción RUN usando &&. Por ejemplo, en lugar de:

```
dockerfile

RUN apt-get update
RUN apt-get install -y curl
```

Es preferible:

```
dockerfile

RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```

  Esto no solo reduce capas, sino que también limpia la caché del gestor de paquetes en la misma capa, disminuyendo el tamaño final de la imagen. En httpd, si no necesitamos instalar nada extra, evitaremos RUN innecesarios.
- **Usar imágenes base mínimas**: Las variantes alpine o slim reducen considerablemente el tamaño de la imagen y, por tanto, la superficie de ataque y el tiempo de descarga. En httpd, httpd:2.4-alpine es una excelente opción para webs estáticas.
- **No incluir secretos en la imagen**: Nunca uses ENV para almacenar contraseñas, claves API u otros datos sensibles, ya que quedarían grabados en las capas de la imagen. En una web Apache con PHP y conexión a MySQL, la contraseña de la base de datos no debe ir en el Dockerfile; se pasará con -e al ejecutar docker run.
- **Añadir metadatos con LABEL**: Puedes documentar la imagen con información como el mantenedor, la versión o la descripción usando LABEL.
  Ejemplo: LABEL maintainer="tu-nombre@ejemplo.com".

7.4. Construcción de la imagen: `docker build`

Una vez escrito el Dockerfile, construimos la imagen con el comando:

docker build -t nombre_imagen:etiqueta .

- -t nombre_imagen:etiqueta: Asigna un nombre y una etiqueta a la imagen resultante (por ejemplo, mi-web:1.0). Si no se especifica etiqueta, se usa latest.
- . (punto final): Indica el **contexto de construcción**, es decir, el directorio cuyos archivos estarán disponibles para las instrucciones COPY y ADD. El Dockerfile se busca por defecto en ese directorio con el nombre Dockerfile.

Ejemplo práctico completo: imagen propia de Apache con nuestra web

Imaginemos que tenemos una web estática sencilla (index.html) y queremos crear una imagen de Apache que ya la incluya. El Dockerfile podría ser:

```
# Imagen base ligera de Apache
FROM httpd:2.4-alpine

# Metadatos de la imagen
LABEL maintainer="tu-nombre@ejemplo.com"
LABEL description="Apache con mi web personalizada"

# Copiar nuestra web al directorio de documentos de Apache
COPY index.html /usr/local/apache2/htdocs/

# (Opcional) Copiar una configuración personalizada de Apache
# COPY httpd.conf /usr/local/apache2/conf/httpd.conf

# Documentar el puerto (informativo)
EXPOSE 80
```

Construiríamos la imagen con:

```
docker build -t mi-web:1.0 .
```

Y la ejecutaríamos con:

```
docker run -d -p 8080:80 --name web-personalizada mi-web:1.0
```

A diferencia de los ejemplos del apartado 5, donde hacíamos docker exec o docker cp para modificar el index.html de un contenedor en ejecución, ahora la web ya forma parte de la imagen: si destruimos el contenedor y creamos otro a partir de mi-web:1.0, la web seguirá estando ahí. Además, podemos compartir esa imagen con otros compañeros o subirla a Docker Hub.

7.5. Publicación de la imagen en Docker Hub

Una vez construida y probada la imagen, podemos compartirla subiéndola a Docker Hub (o a otro registro), igual que hicimos en el apartado 2. Para ello:

Etiquetar la imagen con el formato usuario_dockerhub/nombre_repositorio:etiqueta. Por ejemplo:

```
docker tag mi-web:1.0 miUsuario/mi-web:1.0
```

Iniciar sesión en Docker Hub (si no lo hemos hecho ya):

```
docker login
```

Subir la imagen:

```
docker push miUsuario/mi-web:1.0
```

Tras unos segundos, la imagen estará disponible en tu repositorio de Docker Hub y podrás descargarla desde cualquier equipo con docker pull miUsuario/mi-web:1.0. De este modo, cualquier compañero podrá ejecutar tu Apache personalizado sin necesidad de tener el Dockerfile ni el index.html.

7.6. Ejercicio propuesto

1. Crea un directorio de trabajo y, dentro, un archivo index.html con un mensaje de bienvenida:

   <h1>Hola desde mi Apache personalizado</h1>

2. Crea un Dockerfile que utilice httpd:2.4-alpine como base y copie tu index.html al directorio /usr/local/apache2/htdocs/.
3. Construye la imagen con el nombre mi-apache:v1.
4. Ejecuta un contenedor a partir de ella publicando el puerto 8080 y comprueba con un navegador o con curl que se muestra tu página.
5. Detén y elimina el contenedor, crea uno nuevo a partir de la misma imagen y verifica que tu web sigue estando (a diferencia de lo que ocurría al modificar un contenedor con docker exec).
6. Etiqueta la imagen para tu usuario de Docker Hub y súbela al registro.

8 .- Docker Compose – Orquestación Básica

Docker Compose permite **definir y arrancar contenedores desde un único archivo YAML**. En lugar de escribir varios docker run, describes el stack y lo levantas con un comando. Teniendo las siguientes ventajas:

- Todo el stack documentado en un archivo.
- Un solo comando para levantar/parar todo.
- Crea una red interna donde los servicios se ven por nombre.

```yaml
services:
  nombre_servicio:
    image: ...
    build: ...
    ports:
      - "host:contenedor"
    volumes:
      - origen:destino
    depends_on:
      - otro_servicio

volumes:
  nombre_volumen:

networks:
  nombre_red:
```

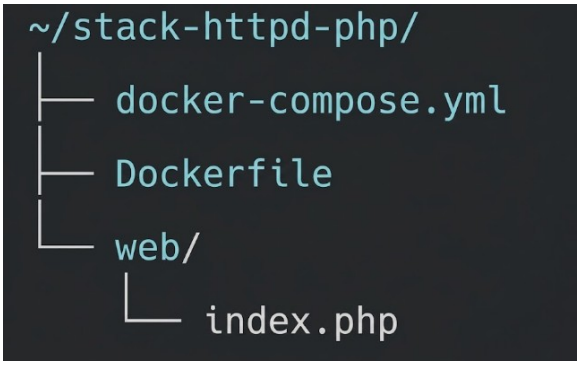
8.1 .- Sintaxis básica

8.2 .- Ciclo de vida

Comando	Descripción
docker compose up -d --build	*Construye (si hay build) y arranca en segundo plano*
docker compose ps	*Ver estado de los contenedores*
docker compose logs -f	*Ver logs en vivo*
docker compose down	*Para y elimina contenedores y red*
docker compose down -v	*Además borra los volúmenes*

8.3 .- Ejercicio práctico: Apache (httpd) + PHP

Para el ejercicio, se va a considerar la siguiente estructura del proyecto:



Paso 1 : Crear el directorio llamdo web

```
mkdir -p ~/stack-httpd-php/web
cd ~/stack-httpd-php
```

y dentro del directorio web, crear el archivo index.php:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Apache + PHP con Docker Compose</title>
</head>
<body>
    <h1>Hola desde Apache + PHP en Docker Compose</h1>
    <p>Hostname del contenedor: <strong><?php echo gethostname(); ?></strong></p>
    <p>Versión de PHP: <strong><?php echo phpversion(); ?></strong></p>
    <p>Fecha y hora del servidor: <?php echo date('Y-m-d H:i:s'); ?></p>
</body>
</html>
```

Paso 2 : Crear el fichero dockerfile

La imagen oficial **httpd no trae PHP.** Partimos de httpd:2.4-alpine y le instalamos PHP.

```
# Imagen base ligera de Apache
FROM httpd:2.4-alpine

# Metadatos
LABEL maintainer="alumno@asir.local"
LABEL description="Apache + PHP sobre httpd:2.4-alpine"

# Instalar PHP y el módulo de integración con Apache
# El paquete php83-apache2 configura automáticamente el handler de PHP
RUN apk add --no-cache php83 php83-apache2

# Copiar la web al directorio de documentos
COPY web/ /usr/local/apache2/htdocs/

# Documentar el puerto (informativo)
EXPOSE 80
```

Nota: en httpd:2.4-alpine el gestor de paquetes es apk (Alpine). Si usaras httpd:2.4 (Debian), sería apt-get update && apt-get install -y php.

Paso 3 : Crear el docker-compose.yml

```
services:
  web:
    build: .                        # construye desde el Dockerfile del directorio actual
    container_name: mi-apache-php
    ports:
      - "8080:80"                   # host:contenedor
    volumes:
      - ./web:/usr/local/apache2/htdocs/   # bind mount: edición en vivo
    restart: unless-stopped
```

Paso 4 : Construir y levantar, debe aparecer la salida indicada

```
docker compose up -d --build
```

```
[+] Building 20.5s (8/8) FINISHED
 => [web 1/4] FROM docker.io/library/httpd:2.4-alpine
 => [web 2/4] RUN apk add --no-cache php83 ...
 => [web 3/4] RUN sed -i ...
 => [web 4/4] COPY web/ /usr/local/apache2/htdocs/
 ✔ Container mi-apache-php   Started
```

Salida Esperada

Paso 5 : Verificar y obtener la salida esperada

```
docker compose ps
```

```
NAME              IMAGE                  SERVICE   STATUS    PORTS
mi-apache-php     stack-httpd-php-web    web       Up        0.0.0.0:8080->80/tcp
```

Salida Esperada

Ahora es el momento de abrir el navegador, con la url : http://localhost:8080 y debería aparecer la página web con el contenido:

> **Hola desde Apache + PHP en Docker Compose**
> Hostname del contenedor: `mi-apache-php`
> Versión de PHP: `8.3.x`
> Fecha y hora del servidor: ...

Paso 6 : Probar el bind mount (edición en vivo)

Edita web/index.php en el host (por ejemplo, cambia el <h1>):

```
sed -i 's/Hola desde Apache + PHP en Docker Compose/¡Modificado en caliente!/' web/index.php
```

Refresca el navegador: el cambio aparece al instante, sin reconstruir la imagen ni reiniciar el contenedor. Esa es la ventaja del bind mount en desarrollo

Paso 7 : Ver logs

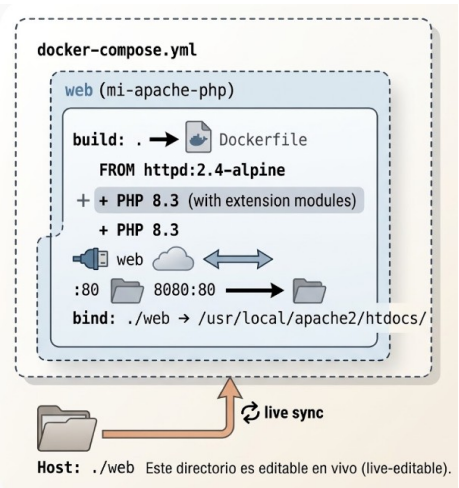
```
docker compose logs -f web
```

Verás las peticiones que llegan a Apache en tiempo real. Útil para depurar.

Paso 8 : Destruir el stack

```
docker compose down
```

Resumiendo los pasos anteriores en un imagen:



8.6 .- Ejercicio Propuesto

1. Crea el stack y comprueba que http://localhost:8080 muestra la página PHP con la versión y el hostname.

2. Modifica web/index.php en el host (cambia el <h1>) y refresca el navegador. ¿Se ve el cambio sin reiniciar? *(Sí, por el bind mount).*

3. Cambia el puerto del host a 9090 en el docker-compose.yml y ejecuta docker compose up -d. Comprueba que ahora responde en http://localhost:9090.

4. Ejecuta docker compose down y luego docker compose up -d --build. ¿Se conserva el contenido? *(Sí, porque está en el host, no en el contenedor).*

5. Añade al index.php un phpinfo() al final y comprueba que se ejecuta correctamente. *(Pista: <?php phpinfo(); ?>).*

6. **[IMPORTANTE]** *¿Y si quiero añadir MariaDB después?*
    1. *Cuando domines este stack, el paso natural es añadir un servicio db (MariaDB) con un volumen con nombre para persistencia y conectar PHP a él usando el nombre del servicio (db:3306) como host. Eso ya es el stack completo del apartado 8.4 anterior. Pero primero conviene tener claro este nivel: httpd + PHP + Compose, que es la base de todo lo demás.*